Western New England University

Digital Commons @ Western New England University

Master's Theses - College of Engineering

College of Engineering

5-2025

Side Channel Analysis of Modern Cryptographic Processors

Srinivas Nittala
Western New England University, sn620094@wne.edu

Follow this and additional works at: https://digitalcommons.law.wne.edu/coetheses

Recommended Citation

Nittala, Srinivas, "Side Channel Analysis of Modern Cryptographic Processors" (2025). *Master's Theses - College of Engineering*. 24.
https://digitalcommons.law.wne.edu/coetheses/24

This Thesis is brought to you for free and open access by the College of Engineering at Digital Commons @ Western New England University. It has been accepted for inclusion in Master's Theses - College of Engineering by an authorized administrator of Digital Commons @ Western New England University.

Side Channel Analysis of Modern Cryptographic Processors

Submitted by

Nittala Satya Surya Lakshmi Vasuki Siva Srinivas

A MS thesis submitted to the faculty of
Western New England University
in partial fulfillment of the
requirements for the Master of Science
in Electrical Engineering

Springfield, MA
May 15, 2025

Keywords: Hardware Security, Cryptographic Algorithms, Firmware Extraction, Key Recovery, Differential Power Analysis

Approved by

Love.K. Sah, Ph.D.	May 14, 2025 Date
Neeraj Magotra, Ph.D.	05/12/25 Date
Sergey Butakov, Ph.D.	May 14, 2025 Date
Dean, College of Engineering	Date

Abstract

In an era of escalating cyber threats, hardware-level attacks have emerged as critical vectors for breaching cryptographic security. This thesis investigates two major classes of hardware-based attacks: firmware retrieval, key recovery and side-channel analysis. The first phase demonstrates practical firmware extraction from an embedded device using tools like CH341a, Binwalk, and Ghidra to reveal cryptographic vulnerabilities and recover encryption keys. The second phase explores power analysis attacks, specifically Differential Power Analysis (DPA), using the ChipWhisperer platform on STM32 and XMEGA microcontrollers to successfully extract AES encryption keys without physical tampering. These experiments underscore the limitations of relying solely on software-based defenses and highlight the ease with which attackers can exploit hardware-level information leakage. The findings advocate for stronger hardware security mechanisms, including secure boot, key masking, and power balancing. Ultimately, this work contributes to the understanding of hardware-rooted cryptographic vulnerabilities and emphasizes the need for holistic, silicon-level security approaches in modern computing systems.

Acknowledgment

I would like to express my sincere gratitude to my advisor**,** Prof. Love K. Sah, Ph.D.**,** for his constant support and invaluable guidance throughout this project. I am also deeply thankful to the Head of Department, Prof. Neeraj Magotra, Ph.D., and all the faculty members for their encouragement and for providing the necessary resources that greatly contributed to the success of this work. Additionally, I would like to extend my heartfelt thanks to my committee member, Prof. Sergey Butakov, Ph.D., for his valuable insights and constructive feedback throughout the course of this research. This journey would not possible without my mother's constant motivation. Lastly, I thank everyone who contributed directly or indirectly to the completion of this thesis.

Table of Contents

Abstract …….i
Acknowledgment……...ii
Table of Contents……...iii
List of Figures……iv
Chapter 1…….…...01
1.1 Introduction……..01
1.2 Why Software Security is not enough ……..02
1.3 Hardware Security……03
1.4 Cryptography and its role in security……03
1.5 Theis Contribution and Organization……...04
Chapter 2 Cryptography……06
2.1 What is Cryptography? 06
2.2 Symmetric Cryptography ……07
2.3 Asymmetric Cryptography ……..…08
2.4 Symmetric key block ciphers……...10
2.4.1 DES (Data Encryption Standard) ...……..11
2.4.1.1 How does DES work? …….…..11
2.4.1.2 Applications of DES …….…13
2.4.2 AES (Advanced Encryption Standard) ……14
2.4.2.1 How does AES work? …….…..15
2.4.2.2 Application of AES ……19
Chapter 3 Firmware Extraction and Key Recovery……21
3.1 Contextual Terms ……21
3.1.1 Firmware Extraction ……21
3.1.2 Executable Linkable Format (ELF) ……22
3.1.3 EEPROM ……..23
3.1.4 SPI (Serial Peripheral Interface) ……..23
3.2 Firmware Extraction ……24
3.3 Firmware Analysis ……..30
3.4 Locating of Decryption Key ……34
Chapter 4 Differential Power Analysis on Cryptographic Processors……39
4.1 Does AES encryption have any security Problems?.....39
4.2 Introduction to Power Analysis ……40
4.3 Instruction to Power differences……..42
4.4 Differential Power Analysis ……46
Chapter 5 Conclusion and Future Work…….……49
References…….……50

List of Figures

Figure 1: Symmetric Encryption [42]……..…..08
Figure 2: Asymmetric Encryption [42] ……..…..09
Figure 3: Working of DES algorithm [41]……..…..13
Figure 4: Working of AES algorithm [40] …….…16
Figure 5: S-box [39]…….…17
Figure 6: Shift rows [38] ……18
Figure 7: Mix Columns [37] …….……...19
Figure 8: ELF header ……...22
Figure 9: EEPROM [36]……..……23
Figure 10: SPI Communication protocol [35] ……24
Figure 11: TP-Link 300Mbps N router [34] ……..…..24
Figure 12: CH341a ROM programmer with SOIC8 clip with jumper wires [33] ……..… 25
Figure 13: Capturing SPI signal using logic analyzer …….…...26
Figure 14: SPI boot process capture …….…..27
Figure 15: SPI protocol data decoding ……...27
Figure 16: CH341a was connected to the EEPROM …….…28
Figure 17: Reading Underway …….…..29
Figure 18: Extracted ".bin" file …….…29
Figure 19: Executed binwalk on ".bin" file ……30
Figure 20 (a): Rising and falling edge of randomness ……...31
Figure 20 (b): Graphical representation of the randomness …….…..31
Figure 21 (a): Extracted sections of the firmware …….32
Figure 21 (b): ".bin" extracted file …….33
Figure 22: Encrypted config files ……33
Figure 23: Ghidra logo [32] …….34
Figure 24 (a): Running search for "decrypt" ……..35
Figure 24 (b): function "dm_decryptFile" ……..35
Figure 25 (a): Decompiled C code ……..36
Figure 25 (b): DES decryption key …….36
Figure 26: Decrypted ".xml" with decryption key …….37
Figure 27 (a): Idle power trace of STM32F303 …….…44
Figure 27 (b): Idle power trace of ATMEL XMEGA ……44
Figure 28 (a): Operational Power trace of STM32F303 …….45
Figure 28 (b): Operational Power trace of ATMEL XMEGA…….45

Figure 29 (a): DPA of a single key bit for the XMEGA …….….47
Figure 29 (b): DPA of a single key bit for the STM32F303…….….47
Figure 30: Recovered AES key from STM32F303 and XMEGA……..……..…….48

Chapter 1

1.1 Introduction

In December 2020, the world witnessed one of the most advanced cyberattacks in history. The SolarWinds supply chain attack compromised over 18,000 organizations, including U.S. federal agencies and Fortune 500 companies, by injecting malicious code into a routine software update [1]. It went undetected for months

Another concerning incident occurred in 2017, the Equifax data breach. It exposed personal information of over 147 million people due to a vulnerability in the Apache Struts web application framework [2]. The attackers didn't require insider knowledge or physical access, just a simple overlooked patch. As banks, health institutions, and governments rushed to tighten software loopholes, the attackers moved faster, always one step ahead.

Disguised as ransomware, NotPetya inflicted approximately $10 billion in damages in 2017 after destroying Ukraine's infrastructure and swiftly spreading throughout the world. It demonstrated how significant systems can be impacted by hackers that spread far from their original location. According to ENISA's Threat Landscape 2023, these kinds of attacks represent important turning points in the evolution of modern cyberwarfare [3]. Strangely enough, it used legitimate software updates to propagate, reiterating how even trusted platforms can be used against users [4].

Then, in 2010 Stuxnet attack was designed to physically sabotage Iran's nuclear centrifuges by manipulating hardware through malware [5]. What set it apart was not just its stealth but its precision. It was one of the first known attacks that bridged the gap between cyber and physical domains. It wasn't just a software breach, it was hardware manipulation, executed remotely.

More recently, the Meltdown and Spectre vulnerabilities revealed in 2018 stunned the global tech community. These were not just bugs in code; they were fundamental flaws in hardware design, allowing attackers to extract sensitive data directly from memory by exploiting speculative execution [6], [7]. These attacks didn't rely on traditional malware or malicious input, they exploited microarchitectural behaviors of processors themselves.

These incidents collectively present a sobering lesson: software security, while essential, is not sufficient on its own. Each of these attacks leveraged software vulnerabilities, but their real power lay in exploiting deep-seated assumptions about trust, privilege, and architecture. Even the most secure software cannot protect against threats that exploit the hardware it runs on.

1.2 Why software security is not enough?

Traditional software security focuses on writing secure code, patching vulnerabilities, and employing runtime defenses like firewalls, antivirus, and intrusion detection systems [8]. These approaches operate at the application and system levels. However, as attacks grow in sophistication, adversaries increasingly bypass these defenses by targeting deeper layers of firmware, microcode, and physical hardware itself [9].

Software security is inherently reactive. It often involves identifying known threats and deploying patches after the fact. Zero-day vulnerabilities, flaws unknown to the software vendor, can exist undetected for years, giving attackers long windows of opportunity [10]. Moreover, if the software stack runs on compromised or untrusted hardware, no amount of software validation can guarantee its integrity.

Additionally, with the proliferation of IoT devices, embedded systems, and edge computing, software is now deployed on billions of devices with constrained resources and minimal update

mechanisms [11]. These devices often lack robust security postures, making them ideal entry points for adversaries.

1.3 Hardware Security

Hardware security refers to protective measures applied directly at the hardware level to defend systems against unauthorized access, data leakage, and tampering. Unlike software security, which can be modified or disabled, hardware security mechanisms are built into the physical components, making them harder to circumvent [12].

Modern hardware security encompasses a wide range of techniques secure boot mechanisms, Physical Unclonable Functions (PUFs), Trusted Platform Modules (TPMs), secure enclaves, side-channel attack mitigation, and anti-tampering designs [13], cryptographic coprocessors. These approaches ensure that even if the software is compromised, the hardware can enforce critical security policies or detect breaches.

The key advantage of hardware-based security is its ability to provide a root of trust, a foundational element that the rest of the system can rely on. For example, secure boot chains ensure that only authenticated and untampered firmware can execute at startup [14]. Hardware security modules (HSMs) protect cryptographic keys from being extracted or misused, even in the event of software compromise [15].

1.4 Cryptography and its role in security

Cryptography has long served as the cornerstone of information security, enabling confidentiality, integrity, and authentication in digital communications. It protects sensitive data through mathematical transformations, rendering information unreadable to unauthorized entities [16].

Public-key cryptography, introduced in the 1970s, revolutionized secure communications by eliminating the need for shared secret keys [17]. Today, cryptographic protocols such as AES, DES, RSA, and ECC are foundational in securing everything from online banking to government communications [18]. Cryptography is hardware-implemented via dedicated modules (HSMs, TPMs), cryptographic accelerators. As cyber threats evolve, robust cryptographic techniques, including encryption, hashing, and key management, remain vital for safeguarding digital trust and preventing unauthorized access.

1.5 Thesis Contribution and Organization

This thesis work investigates cryptographic vulnerabilities in hardware systems, focusing on firmware extraction, reverse engineering, and side-channel attacks to analyze key extraction techniques. By systematically evaluating these attack vectors, we identify weaknesses in hardware-based security mechanisms. The findings contribute to the development of effective countermeasures, enhancing hardware resilience against cryptographic breaches. In an era of escalating cyber threats, securing hardware at the silicon level is critical to establishing a trustworthy computing foundation.

The remainder of this thesis is structured as follows:

- **Chapter II** provides foundational background on cryptographic algorithms, hardware security primitives, and threat models relevant to key extraction.
- **Chapter III** details *Thesis Work I*: firmware extraction for key extraction. We demonstrate a practical methodology using tools like Binwalk and Ghidra to analyze chip firmware, exposing vulnerabilities in cryptographic implementations.

- **Chapter IV** presents *Thesis Work II*: advanced side-channel analysis (SCA) via Differential Power Analysis (DPA). We target modern processors (STM32, XMEGA) with internal key storage, showcasing key extraction *without physical probes* by exploiting power leakage.
- **Chapter V** discusses broader implications, mitigation strategies, and future research directions for securing hardware against cryptographic attacks.

Chapter 2

Cryptography

2.1 What is Cryptography?

In simple terms, the art of converting information by transforming it into an unreadable format is called cryptography.

It enables secure communication by ensuring that only intended recipients can access the message, typically through the use of a decryption key. This concept of safeguarding information through transformation has ancient roots. For instance, the Spartans utilized the scytale, a cylindrical cipher device, while Julius Caesar employed a substitution method now known as the Caesar cipher [19]. A major milestone in cryptographic history occurred during World War II with the development of the Enigma machine, which was eventually decrypted by Alan Turing and his team—an achievement that significantly influenced the outcome of the war.

In the modern digital era, cryptography is essential in securing various online activities, such as e-commerce, messaging, and electronic banking. Advanced algorithms like the Advanced Encryption Standard (AES) and the Rivest-Shamir-Adleman (RSA) protocol are widely implemented to ensure data confidentiality and integrity. Cryptographic systems are broadly categorized into two types: symmetric and asymmetric. Symmetric encryption uses a single shared key for both encryption and decryption, whereas asymmetric encryption utilizes a pair of keys, public and private, for secure communication.

2.2 Symmetric Cryptography

Symmetric cryptography is an encryption technique in which the same secret key is used for both encryption and decryption. This can be visualized as a secured box that is locked with a unique key only the sender and receiver possess this identical key. The sender encrypts the message using the key, and the receiver uses the same key to decrypt and read the original message.

In terms of performance, symmetric encryption is significantly faster and more efficient than asymmetric encryption, as it requires fewer CPU cycles. This makes it particularly suitable for resource-constrained environments and for transmitting large volumes of data securely. However, a major limitation is the secure distribution of the secret key. If the key is intercepted during transmission over an insecure channel, the confidentiality of the entire communication is compromised [19].

Symmetric cryptography is also referred to as secret key, private key, or conventional key cryptography. Among the most well-known symmetric encryption algorithms are the Data Encryption Standard (DES) and the Advanced Encryption Standard (AES), with AES being the modern standard due to its stronger security features.

Figure 1 below illustrates the fundamental working principle of symmetric encryption, showing how the same key is used at both ends of communication to ensure confidentiality.

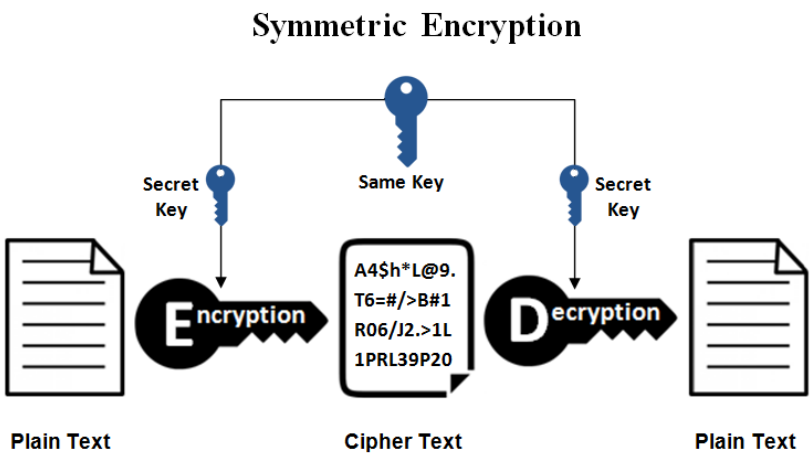


Figure 1: Symmetric Encryption [42]

2.3 Asymmetric Cryptography

Asymmetric cryptography, also known as public key cryptography, uses a pair of keys: a public key for encryption and a private key for decryption. The public key can be shared openly, while the private key remains confidential. Anyone can encrypt a message using the public key, but only the intended recipient can decrypt it with their private key, ensuring secure communication.

This method provides confidentiality, integrity, and authentication. It is more computationally intensive than symmetric cryptography due to complex mathematical problems like integer factorization and discrete logarithms [19].

A key application is the secure exchange of symmetric keys over untrusted channels, as seen in SSL/TLS protocols. Here, the symmetric key is encrypted with the receiver's public key and decrypted using their private key, establishing a secure session.

Depending on the use confidentiality or digital signatures either the public or private key can be used for encryption or verification. Common algorithms include RSA and DSA, widely used in modern security systems.

As shown in Figure 2, asymmetric encryption ensures that only the holder of the private key can decrypt data encrypted with the corresponding public key [19].

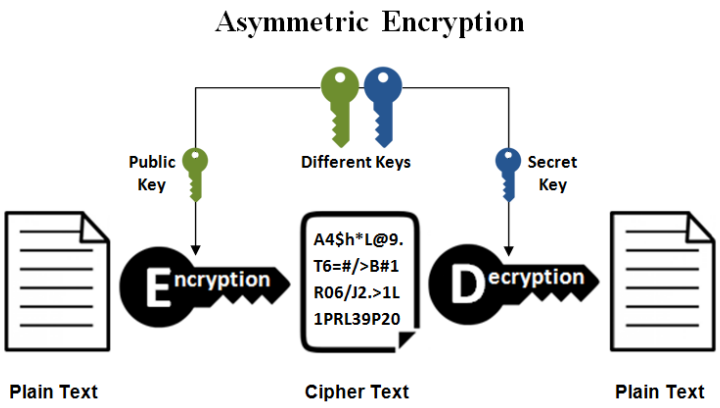


Figure 2: Asymmetric Encryption [42]

2.4 Symmetric key block ciphers

Data encryption is a secure lock to protect digital information and remains a key element in modern cybersecurity. Among the most widely discussed encryption algorithms are the Data Encryption Standard (DES) and the Advanced Encryption Standard (AES) [19].

DES was once a widely used encryption method, relying on a 56-bit key. While this was considered secure in the past, advances in computing power have made DES vulnerable to brute-force attacks, leading to its decline in practical use.

To address these limitations, AES was introduced as a more robust and efficient successor. AES supports key lengths of 128, 192, and 256 bits, providing a significantly higher level of security. Today, AES is the preferred standard for securing various applications, including online banking and encrypted communications.

The shift from DES to AES highlights the need for encryption technologies to advance alongside emerging cyber threats. As computing capabilities grow, so must the strength and complexity of cryptographic algorithms. This ongoing evolution drives researchers to continuously develop improved encryption techniques.

2.4.1 DES (Data Encryption Standard)

The Data Encryption Standard (DES) is an early symmetric encryption algorithm developed to secure digital data. It operates on 64-bit blocks of plaintext, converting them into 64-bit blocks of ciphertext using a 56-bit key. However, due to advances in computational power, DES is now considered insecure, as it can be broken in a relatively short time using brute-force attacks.

DES does not perform encryption in a single step; instead, it uses a fixed series of operations known as "rounds." Specifically, DES applies 16 identical rounds of processing to add multiple layers of complexity to the encryption process. Unlike modern standards like the Advanced Encryption Standard (AES), which support variable key lengths and numbers of rounds, DES uses a fixed key size and round count [19].

2.4.1.1 How does DES work?

The Data Encryption Standard (DES) is a symmetric key block cipher designed to protect the confidentiality of digital information. It operates by dividing the input message into 64-bit blocks and encrypting each block separately, making it a block cipher, a method that handles fixed-size chunks of data, unlike stream ciphers, which encrypt data bit by bit.

DES is based on an earlier cipher called Lucifer and uses a combination of substitution (replacing bits) and permutation (rearranging bits) techniques to obscure the original data. At the beginning of the encryption process, DES applies an Initial Permutation (IP) to the 64-bit block, rearranging its bits in a predefined order.

This permuted block is then split into two 32-bit halves: the Left Plain Text (LPT) and the Right Plain Text (RPT). The encryption process proceeds through 16 rounds of transformation, each involving the following steps:

1) **Key Transformation**: A different 48-bit round key is generated from the original 56-bit key for each round.
2) **Expansion Permutation**: The 32-bit RPT is expanded to 48 bits to match the size of the round key.
3) **XOR Operation**: The expanded RPT is combined with the round key using the XOR operation.
4) **S-Box Substitution**: The result is compressed back to 32 bits through substitution boxes (S-boxes), which introduce non-linearity and confusion.
5) **P-Box Permutation**: The 32-bit output is permuted again using a fixed pattern known as the P-box.

After each round, the LPT and RPT are swapped. This process is repeated 16 times. At the end of the final round, the two halves are recombined, and a Final Permutation (FP) is applied to produce the 64-bit ciphertext [19].

Decryption is achieved by reversing the encryption steps using the same key but applying the round keys in reverse order. The overall working of DES is illustrated in the figure below.

Figure 3: Working of the DES algorithm [41]

2.4.1.2 Applications of DES

- **Use in Pseudorandom Number Generation (PRNG):**

  DES is utilized in some cryptographic systems to generate hard-to-predict pseudorandom numbers [19].

- **Application in Legacy or Low-Security Systems:**

  DES remains in use in older software/hardware where high security is not critical and upgrades are impractical due to its simplicity and speed.

- **Foundation for Triple DES (3DES):**

  DES inspired the development of 3DES, which applies DES three times with separate 56-bit keys to enhance security (total key length: 168 bits) [19], [20].

2.4.2 AES (Advanced Encryption Standard)

The Advanced Encryption Standard (AES) is a widely used symmetric encryption algorithm designed to protect data from unauthorized access using a secret key. Depending on the key length, it is categorized as AES-128, AES-192, or AES-256 [19].

In the 1970s, the U.S. government required a secure encryption method for sensitive data, which led to the adoption of the Data Encryption Standard (DES). However, as computational power increased, DES became vulnerable. It was successfully broken in 1997, and by 1999, it could be cracked in just over 22 hours. By 2021, DES could be broken in under 5 minutes [21].

To address these weaknesses, Triple DES was introduced, but it was computationally inefficient. In response, the National Institute of Standards and Technology (NIST) initiated a public competition in 1997 to identify a stronger replacement for DES. Fifteen encryption algorithms were submitted by 1998. After extensive evaluation, NIST selected the Rijndael algorithm in 2001 as the new encryption standard AES.

AES gained popularity due to its efficiency, platform compatibility, and availability without licensing fees, making it suitable for widespread use.

2.4.2.1 How does AES work?

The Rijndael algorithm, which forms the foundation of the Advanced Encryption Standard (AES), is a symmetric key cryptographic method. This means it uses the same key for both encryption and decryption processes, ensuring secure communication between sender and receiver.

AES is widely adopted due to its strong security and high performance. It is based on a block cipher structure, where the input data (plaintext) is divided into fixed-size blocks and encrypted using multiple transformation steps. Each of these steps is designed to obscure the relationship between the plaintext and the ciphertext effectively.

Internally, AES operates on a 4×4 grid of bytes known as the "State", totaling 16 bytes. The number of encryption rounds depends on the key size: 10 rounds for a 128-bit key, 12 rounds for a 192-bit key, and 14 rounds for a 256-bit key [19]. Each round applies four core operations:

- SubBytes: A non-linear byte substitution using an S-box.
- ShiftRows: A cyclic shifting of bytes in each row.
- Mix Columns: A mixing operation that combines the bytes of each column.
- AddRoundKey: A bitwise XOR between the State and a round key derived from the main encryption key.

The round keys are generated through a process known as the Rijndael key schedule, which expands the original key into multiple round-specific subkeys. Decryption follows the reverse of these operations using the same key. The structure of AES is a substitution-permutation network, where layers of substitution (S-box) and permutation (P-box) operations ensure both confusion

and diffusion, enhancing cryptographic strength. Due to its efficiency and robustness, AES is the standard encryption algorithm used globally in applications ranging from secure communications to banking systems. Figure 4 provides a high-level overview of how AES encryption works.

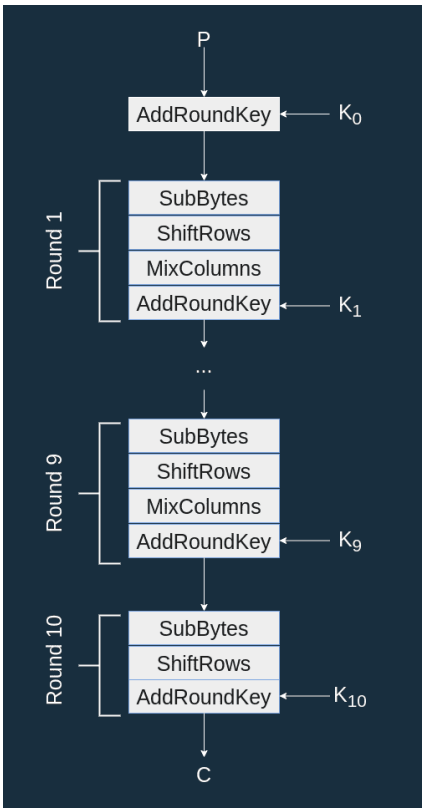


Figure 4: Working of AES algorithm [40]

1. Sub Bytes

The SubBytes operation is the initial step in each round of the AES encryption process. In this step, each byte of the input data is substituted with a different byte according to a fixed substitution box, known as the S-box. This substitution is not random; it uses a well-defined table that maps each byte to another based on its hexadecimal value. For instance, if a byte has a hexadecimal value of 84, AES uses the 8th row and 4th column of the S-box to find the corresponding value, such as 5F, which then replaces the original byte. This process enhances confusion in the cipher, making it more resistant to cryptanalysis. The following figure 5 illustrates how a byte is substituted using the S-box in the SubBytes step.

		y															
		0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
x	0	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
	1	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
	2	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
	3	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
	4	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
	5	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
	6	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
	7	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
	8	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
	9	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
	a	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
	b	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
	c	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
	d	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
	e	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
	f	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

Figure 5: S-Box [39]

2. Shift Rows

In the AES algorithm, the ShiftRows transformation rearranges the bytes in each row of the state matrix to enhance data diffusion. Specifically, the first row remains unchanged, while the second, third, and fourth rows are circularly shifted to the left by one, two, and three positions, respectively. This operation can be compared to shuffling cards, as it helps distribute the data more evenly across the columns, making any identifiable patterns in the encrypted message more difficult to trace. A visual representation of this transformation is shown in Figure 6.

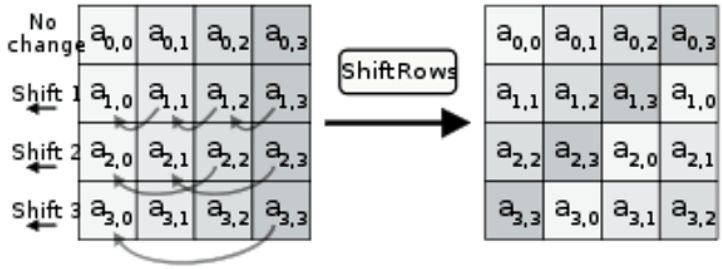


Figure 6: Shift rows [38]

3. Mix Columns

In the AES encryption process, each column of the state matrix undergoes a transformation known as Mix Columns. This operation involves matrix multiplication over a finite field (specifically, the Galois Field GF(2^8)). The goal is to securely combine all four bytes of each column, ensuring strong diffusion of data. Though it involves XOR operations and arithmetic in a specialized mathematical domain, its purpose is straightforward: to thoroughly mix the column data to enhance security. Below is a visual representation of the AES Mix Columns operation.

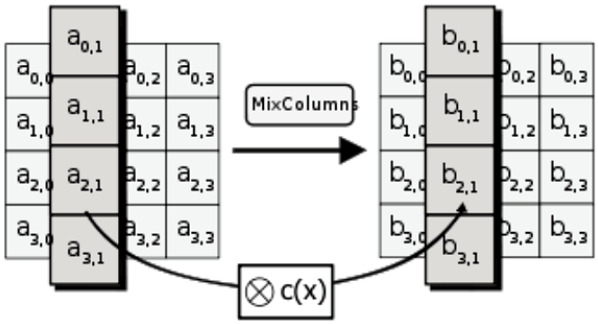


Figure 7: Mix Columns [37]

4. Add Round Key

In the AES encryption process, one of the key operations is the AddRoundKey step. During this step, the data from the previous operation is combined using a bitwise XOR with a portion of the secret key, known as the round key. These round keys are derived from the original key through a process called the key schedule, which follows the Rijndael algorithm. This operation is performed at the beginning of the encryption, after every round, and at the end. Following this, AES returns to the SubBytes step and continues the sequence of four operations until all rounds are finished.

2.4.2.2 Applications of AES

- Cloud platforms like Google Drive use AES-256 encryption to protect uploaded files.
- SSDs (e.g., Samsung) apply AES-256 to secure stored data, making it unreadable without the decryption key even if the drive is removed.
- Mobile apps such as WhatsApp and Facebook encrypt messages using AES, ensuring only the intended recipient can read them [28].
- AES is also used in IoT devices, self-encrypting software, and secure hard drives, typically with 128-bit or 256-bit keys, depending on the security needs [19].

- VPN services rely on AES to encrypt internet traffic, keeping user activity private and hidden from external observers [28].

Chapter 3

Firmware Extraction and Key Recovery

These are the notes I took while experimenting with basic hardware exploitation. Many real-world attacks are successfully carried out using such fundamental techniques. One key advantage is that they can be applied to almost any IoT device. If an attacker already has access to these basic-level methods, more advanced techniques may not even be necessary. Before we begin, let’s review some terms to understand the context better.

3.1 Contextual Terms

3.1.1 Firmware Extraction

Firmware extraction is one of the main issues in hardware security. The low-level software that regulates hardware operations is called firmware. It functions similarly to a gadget's brain. Before the primary operating system launches, it provides instructions to chips and processors. Attackers frequently attempt to remove this firmware in order to examine it, identify any vulnerabilities, and maybe change it for nefarious ends. This could make it possible for someone to take over a system or steal confidential information, which makes it risky. Unfortunately, the firmware of many devices lacks robust protection, particularly older models. The firmware may be altered or reverse-engineered to contain malware once it has been extracted. It's critical to comprehend this procedure in order to enhance security in upcoming products.

Using techniques like secure boot procedures, cryptographic signature, and restricting access to authorized users are all necessary for firmware protection. These safeguards are absent from many devices, particularly older models, which makes them more vulnerable to attacks.

3.1.2 Executable Linkable File Format (ELF)

when utilizing standard Linux systems. ELF (Executable Linkable File Format) [22] is a particular file format that we frequently encounter. These formats are well-structured and include all of the operating system's information, including linking details, memory layout, program instructions, and more.

In IOT devices or microcontrollers, the scenario will be different. Instead of ELF files, we will encounter raw binary files.

ELF instructs the kernel how to create a process and assign segments into memory. The ELF header and program header table are loaded by the kernel before the program is executed. The portions designated as LOAD are then loaded into memory, and it determines whether an interpreter is necessary or not. Before moving further, just take a look at ELF in Figure 8.

```
lone@lone-HP-Laptop-15s-fr2xxx:~/_tplink_router.bin.extracted/squashfs-root/bin$ hd busybox | head
00000000  7f 45 4c 46 01 01 01 00  00 00 00 00 00 00 00 00  |.ELF............|
00000010  02 00 08 00 01 00 00 00  e0 26 40 00 34 00 00 00  |.........&@.4...|
00000020  c8 fb 03 00 07 10 00 70  34 00 20 00 08 00 28 00  |.......p4. ...(.|
00000030  1a 00 19 00 06 00 00 00  34 00 00 00 34 00 40 00  |........4...4.@.|
00000040  34 00 40 00 00 01 00 00  00 01 00 00 05 00 00 00  |4.@.............|
00000050  04 00 00 00 03 00 00 00  34 01 00 00 34 01 40 00  |........4...4.@.|
00000060  34 01 40 00 14 00 00 00  14 00 00 00 04 00 00 00  |4.@.............|
00000070  01 00 00 00 01 00 00 00  00 00 00 00 00 00 40 00  |..............@.|
00000080  00 00 40 00 a8 ec 03 00  a8 ec 03 00 05 00 00 00  |..@.............|
00000090  00 00 01 00 01 00 00 00  00 f0 03 00 00 f0 44 00  |..............D.|
lone@lone-HP-Laptop-15s-fr2xxx:~/_tplink_router.bin.extracted/squashfs-root/bin$
```

Figure 8: ELF header

3.1.3 EEPROM

Similar to flash memory, electrically erasable programmable read-only memory, or EEPROM, retains data even if the power is turned off or resumed. EEPROM has substantially fewer storage pages as little as one byte than flash memory. Because of this, it's wee suited for storing little bits of information like times, dates, device settings, password hashes, usernames, and serial numbers. In embedded systems like microcontrollers and Internet of Things devices, engineers frequently utilize it due to its ability to manage small data quantities. An EEPROM chip is photographed in Figure 9.

Figure 9: EEPROM [36]

3.1.4 SPI (Serial Peripheral Interface)

SPI (Serial Peripheral Interface) is widely used in embedded systems, which allows components like microcontrollers, displays, and memory chips to communicate [23]. It uses at least three lines:

- One for sending data to the device (MOSI).
- Second for receiving data (MISO).
- The third is the clock line (SCK) to stay synchronized.

If there are multiple devices, a chip select line (SS) is needed to pick which one to communicate with. SS is usually kept enabled all the time. When there is a troubleshooting, we'll look at ways to tap into these signals to see what's going on. Figure 10 shows the protocol schematic of SPI.

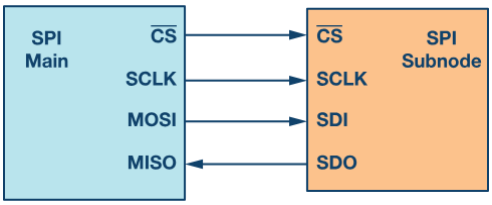


Figure 10: SPI communication protocol [35]

3.2 Firmware Extraction

I used a TP-Link router as a target device to demonstrate the basic hardware exploitation.

Device: TP-Link 300Mbps Wireless N router

Model: TL-WR841N(US)

FCC ID: 2AXJ4WR841NV14

Refer to Figure 11 for an example of the device.

Figure 11: TP-Link 300Mbps Wireless N router [34]

To extract the firmware from the TP-Link router, I used a method that involves reading directly from the memory chip (ROM). For The following tools and steps were required:

- CH341a ROM programmer - It's a very inexpensive tool that interacts with the memory chip to read its contents.
- SOCI8 Clip with Jumper Wires - This tool allows you to connect the CH341a programmer to the router's ROM chip without the need to remove it from the circuit board.
- Target Device - TP-Link router is our target device.

Refer to Figure 12 for CH341a with SOIC8 clip.

Figure 12: CH341a ROM programmer with SOIC8 clip with jumper wires [33]

During OSINT research, I discovered that the router's ROM uses the SPI (Serial Peripheral Interface) protocol to communicate with the microcontroller. I used a logic analyzer to verify that communication was taking place between the processor and the ROM chip. For software, we used PulseView (with Sigrok) to visualize the signals [24]. These tools allow us to observe the data flow between the processor and ROM, which is crucial for analyzing or reverse-engineering the firmware.

Here's how the connections are made:

- **CH0 – CS (Chip Select)**: This signals the flash chip when it should be active and take notice.
- **Ground (Vss) -** Is just to complete the circuit and make sure all signals have a common reference.
- **CH1 – DO (Data Out)**: This is where the flash chip sends data *to* the processor.
- **CH2 – CLK (Clock)**: This keeps everything in sync. Data can be transferred each time the clock cycles.
- **CH3 – DI (Data In)**: This is how the processor sends commands or data to the flash chip.

After all connections are made and the system is powered up, Sigrok starts capturing the SPI signals. Waveforms will show up on all four channels, confirming whether the setup is working correctly. This approach is frequently used for debugging and validating embedded system designs during the early development phases. Refer to Figure 13 for a visual representation of how the setup was done and Figure 14 for a SPI boot-up capture.

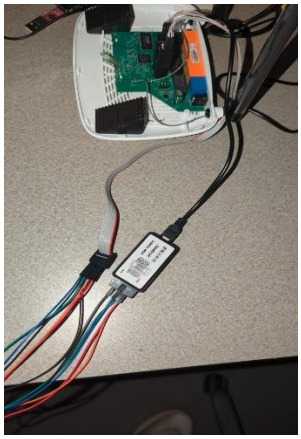


Figure 13: Capturing SPI signal using logic analyzer

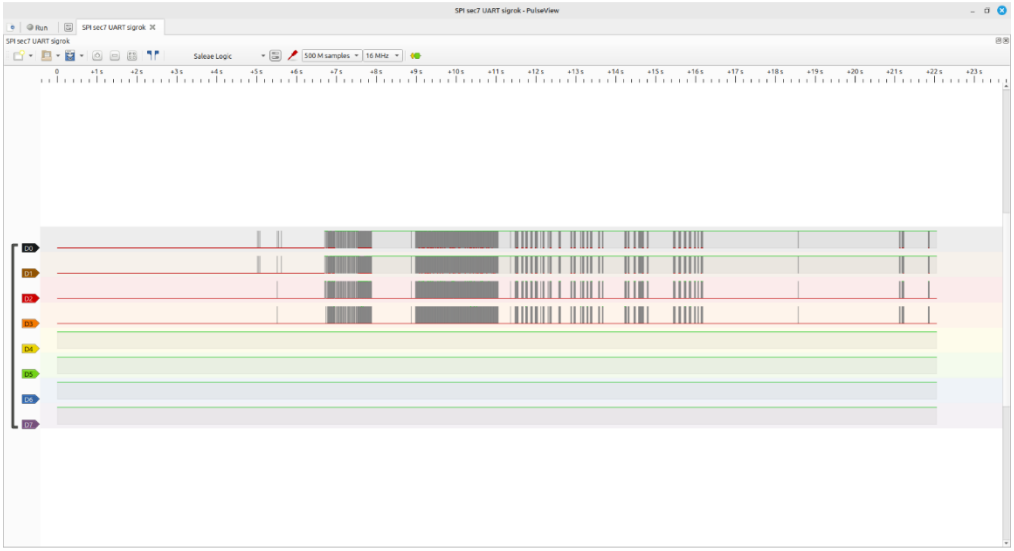


Figure 14: SPI boot process capture

When you zoom in on the waveforms, you'll notice that each signal has its unique pattern, with each line serving a specific function. For example, the CLK (clock) line should appear smooth and regular, much like a metronome's ticks. It should not be erratic or random. A stable clock signal suggests that the system is functioning properly, with data being sent and received as expected. See Figure 15 for the clear pattern in the signals.

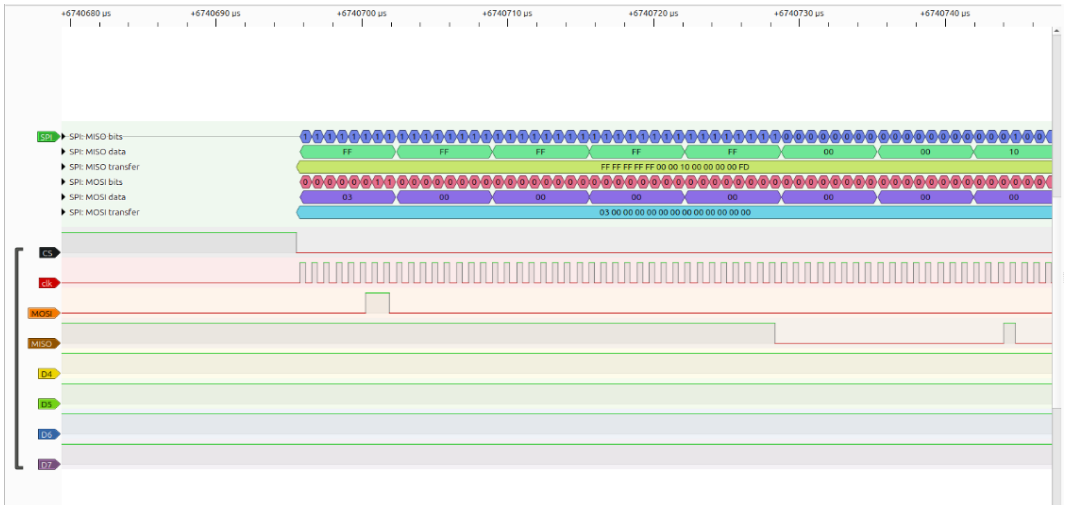


Figure 15: SPI protocol Data Decoding

Some engineers use this method to search for vulnerabilities or reverse-engineer firmware, particularly in secure or critical devices. Capturing the boot process can reveal security issues, such as unencrypted boot sequences or weak authentication

In summary, you use a logic analyzer and Sigrok to monitor the communication between the processor and flash memory over SPI during boot. By properly setting up the equipment and examining the waveforms, you can obtain important insights into the device's internal workings. After confirming that the SPI flash was working correctly, a CH341a flash programmer was connected to the flash ROM according to the pinout in the datasheet to read the full contents of the flash ROM. Refer to Figure 16 for a visual representation of how the setup was done.

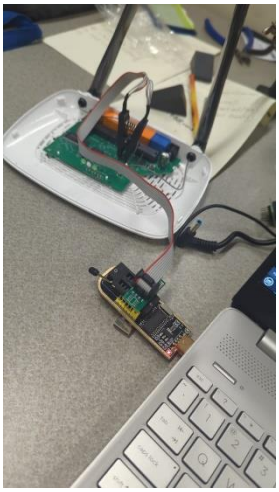


Figure 16: CH341a was connected to the EEPROM

The firmware was subsequently extracted from the ROM using flashrom by executing the following command:

flashrom -p ch341a_spi -r tplink_router.bin

Refer to Figure 17 for the process of reading the firmware, which was in progress.

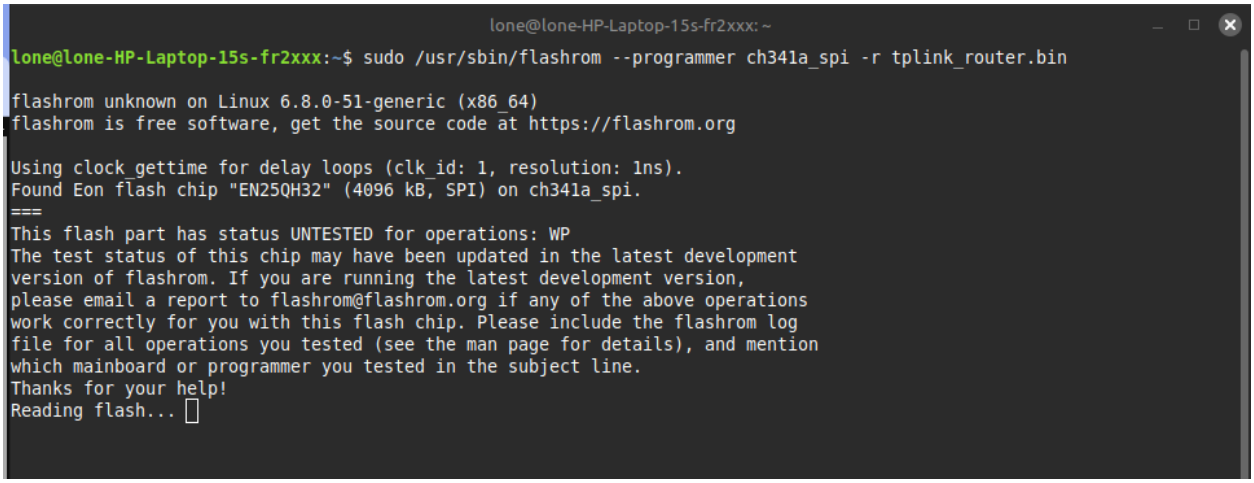


Figure 17: Reading Underway

The command reads the entire flash ROM and saves it as a binary file called “tplink_router.bin”. It is important to turn off the router during the extraction process to prevent bus conflicts or damage to the SPI flash chip. Refer to Figure 18, how the extracted “.bin” file looks like.

Figure 18: Extracted “.bin” file

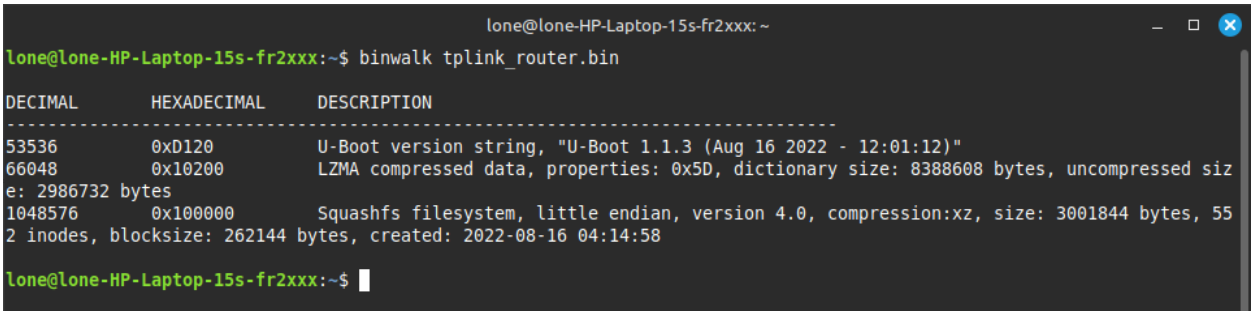
3.3 Firmware Analysis

Binwalk is a command-line tool that is useful for analyzing binary files, particularly firmware [25]. If you have firmware from a vendor but can't physically access the device, Binwalk is an excellent tool to explore its contents. The open-source version has been available for some time and is widely used in the security and reverse-engineering communities.

The tool helps you quickly identify which parts of the file are in use, which areas might be empty, and which sections could be compressed or encrypted. It can also assist in identifying the processor type and extracting known file types.

Two of the most useful options are -E, which analyzes the file's randomness (entropy) and generates an entropy graph, and -e, which attempts to automatically extract known files from the binary.

Run, binwalk tplink_router.bin, hit enter.



```
lone@lone-HP-Laptop-15s-fr2xxx: ~
lone@lone-HP-Laptop-15s-fr2xxx:~$ binwalk tplink_router.bin

DECIMAL       HEXADECIMAL     DESCRIPTION
--------------------------------------------------------------------------------
53536         0xD120          U-Boot version string, "U-Boot 1.1.3 (Aug 16 2022 - 12:01:12)"
66048         0x10200         LZMA compressed data, properties: 0x5D, dictionary size: 8388608 bytes, uncompressed siz
e: 2986732 bytes
1048576       0x100000        Squashfs filesystem, little endian, version 4.0, compression:xz, size: 3001844 bytes, 55
2 inodes, blocksize: 262144 bytes, created: 2022-08-16 04:14:58

lone@lone-HP-Laptop-15s-fr2xxx:~$
```

Figure 19: Executed binwalk on ".bin" file

The tool will identify three distinct components in this binary file. The first is U-boot at 53536, followed by LZMA compressed data at 66048, and then squashfs at 1048576. Next, let's analyze the entropy of the binary file extracted from the CPU (ROM). Refer Figure 19.

Run, binwalk –E tplink_router.bin, hit enter.

```
lone@lone-HP-Laptop-15s-fr2xxx:~$ binwalk -E tplink_router.bin

DECIMAL       HEXADECIMAL     ENTROPY
--------------------------------------------------------------------------------
0             0x0             Falling entropy edge (0.531249)
34816         0x8800          Falling entropy edge (0.849541)
49152         0xC000          Falling entropy edge (0.837666)
67584         0x10800         Rising entropy edge (0.987319)
1040384       0xFE000         Falling entropy edge (0.841460)
1048576       0x100000        Rising entropy edge (0.982279)
4048896       0x3DC800        Falling entropy edge (0.807424)
4063232       0x3E0000        Rising entropy edge (0.989275)
4104192       0x3EA000        Falling entropy edge (0.298464)
```

Figure 20 (a): Rising and falling edge of randomness

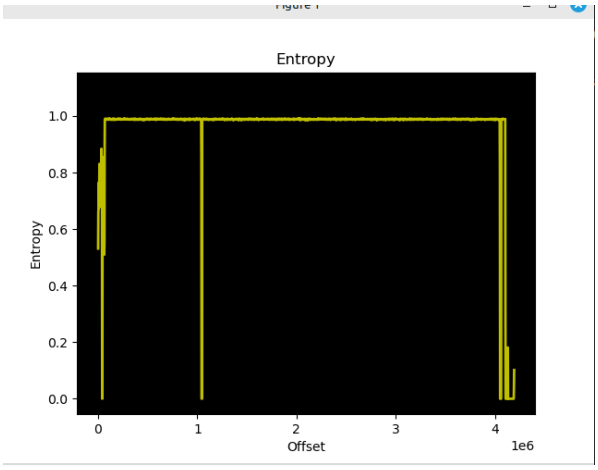


Figure 20 (b): Graphical representation of the randomness

After running the command “binwalk -E tplink_router.bin”, look at the entropy graph to assess the randomness of the file contents. Entropy indicates how random the data is; a value close to 1 suggests the data is very random and likely compressed or encrypted. A value around 0.5 indicates more regular data, likely uncompressed. In Figure 20 (a), witness the randomness of the data. And in Figure 20 (b), see the graphical representation of the randomness of the data.

After that, I used Binwalk again, to automatically pull out the files hidden inside the firmware. To do this, run this command:

binwalk -e tplink_router.bin and hit enter. Binwalk scans through the firmware and extracts anything it can recognize, like file systems or compressed data. In Figure 21 (a) you can see various sections that were extracted, and then in Figure 21(b) extracted file looks like.

```
lone@lone-HP-Laptop-15s-fr2xxx:~$ binwalk -e tplink_router.bin

DECIMAL       HEXADECIMAL     DESCRIPTION
--------------------------------------------------------------------------------
53536         0xD120          U-Boot version string, "U-Boot 1.1.3 (Aug 16 2022 - 12:01:12)"
66048         0x10200         LZMA compressed data, properties: 0x5D, dictionary size: 8388608 bytes, uncompressed siz
e: 2986732 bytes

WARNING: Extractor.execute failed to run external extractor 'sasquatch -p 1 -le -d 'squashfs-root-0' '%e'': [Errno 2]
No such file or directory: 'sasquatch', 'sasquatch -p 1 -le -d 'squashfs-root-0' '%e'' might not be installed correctl
y

WARNING: Extractor.execute failed to run external extractor 'sasquatch -p 1 -be -d 'squashfs-root-0' '%e'': [Errno 2]
No such file or directory: 'sasquatch', 'sasquatch -p 1 -be -d 'squashfs-root-0' '%e'' might not be installed correctl
y

WARNING: Symlink points outside of the extraction directory: /home/lone/_tplink_router.bin.extracted/squashfs-root/etc
/resolv.conf -> /var/tmp/resolv.conf; changing link target to /dev/null for security purposes.

WARNING: Symlink points outside of the extraction directory: /home/lone/_tplink_router.bin.extracted/squashfs-root/etc
/TZ -> /var/tmp/TZ; changing link target to /dev/null for security purposes.

WARNING: Symlink points outside of the extraction directory: /home/lone/_tplink_router.bin.extracted/squashfs-root/etc
/passwd -> /var/passwd; changing link target to /dev/null for security purposes.
1048576       0x100000        Squashfs filesystem, little endian, version 4.0, compression:xz, size: 3001844 bytes, 55
2 inodes, blocksize: 262144 bytes, created: 2022-08-16 04:14:58
```

Figure 21 (a): extracted sections of the firmware

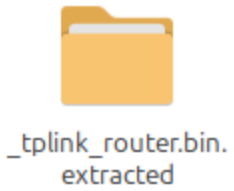


Figure 21 (b): ".bin" extracted file

Our main focus is on the squashfs section of the firmware, as it holds the router's file system. Inside the /etc directory, we discovered several noteworthy files, including two encrypted XML files—default_config.xml and reduced_data_model.xml—that seem to be configuration files. While searching the entire squashfs for these filenames, one file caught our attention: libcmm. so. This shared object file might be interacting with the encrypted configuration files, potentially reading or decrypting them. In Figure 22, you can see the encrypted config files.

```
lone@lone-HP-Laptop-15s-fr2xxx:~/_tplink_router.bin.extracted/squashfs-root/lib$ strings -f * | grep ".xml"
libcmm.so: xml_delTree
libcmm.so: xml_unescapeString
libcmm.so: xml_escapeString
libcmm.so: xml_getAttr
libcmm.so: xml_parseString
libcmm.so: xml_findPrev
libcmm.so: <?xml version="1.0"?>
libcmm.so: ?xml
libcmm.so: Populate xml node %s error.
libcmm.so: /etc/default_config.xml
libcmm.so: Valid values array doesn't have name attribute in xml file.
libcmm.so: The xml node is not a parameter node.
libcmm.so: /etc/reduced_data_model.xml
libcmm.so: Parse xml file %s failed.
libcmm.so: can not open xml file %s!, about to open file %s
libcmm.so: can not open xml file %s!
libcmm.so: can not get xml file %s's stat
libcmm.so: read xml file(%s) error.
```

Figure 22: encrypted config files.

3.4 Locating of Decryption Algorithm and Key

Ghidra is a free, open-source reverse engineering tool developed by the National Security Agency (NSA) [26]. It was first presented to the public at the RSA Conference in March 2019, with its source code released in April of the same year. Ghidra provides a wide range of features, making it a comprehensive and well-polished tool, which is uncommon for open-source software. You can see the GHIDRA logo in Figure 23 below.

Figure 23: GHIDRA logo [32]

I used Ghidra to analyze a binary and was able to extract the decryption key by examining the code flow and memory references. We are just scratching the surface of GHIDRA.

Running a search for “decrypt” in GHIDRA revealed a function called “dm_decryptFile” that was then investigated further. Figure 24 (a) is about running the search for “decrypt” and Figure 24 (b) is about the revealed function “dm_decryptFile”.

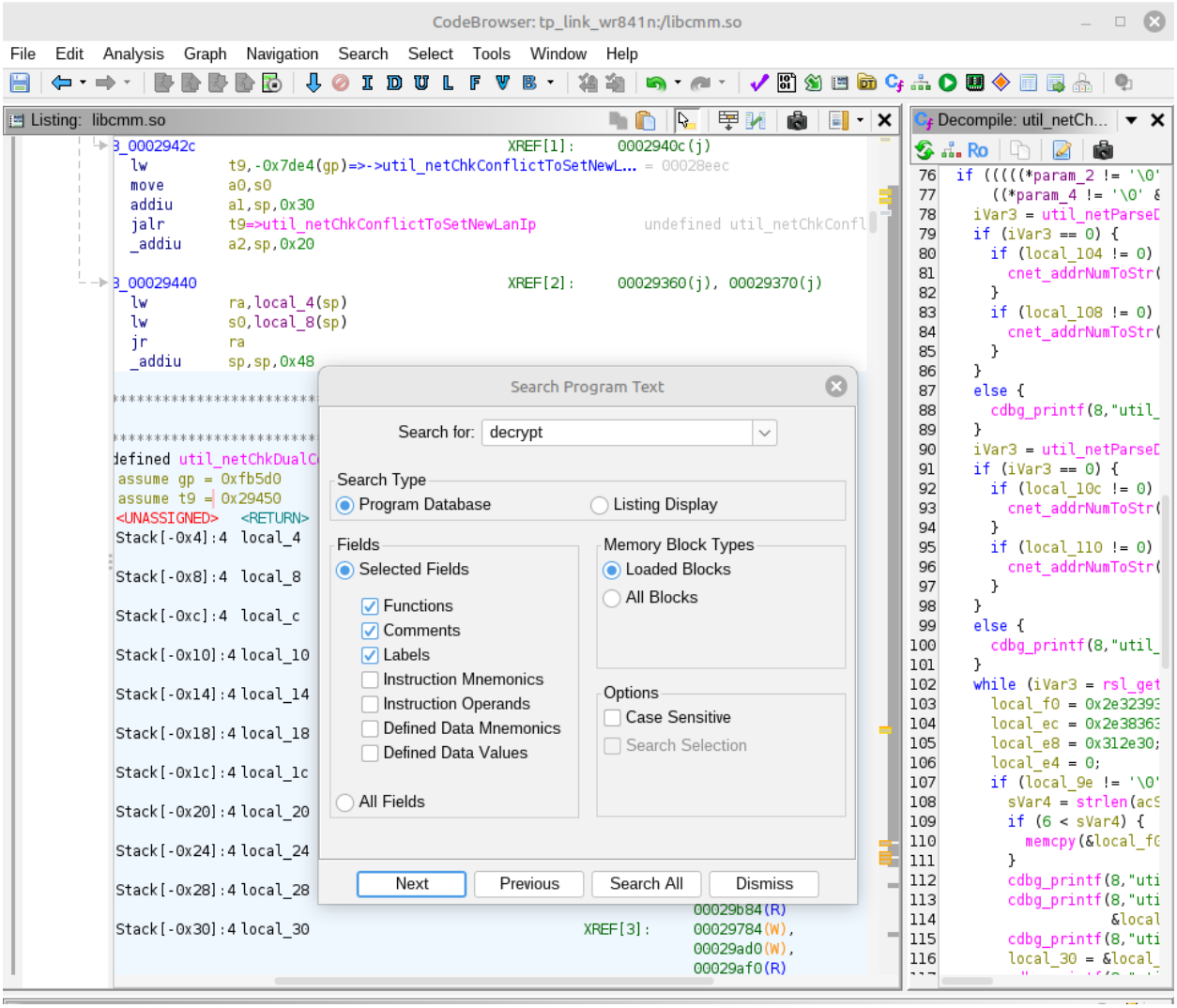


Figure 24 (a): running search for "decrypt"

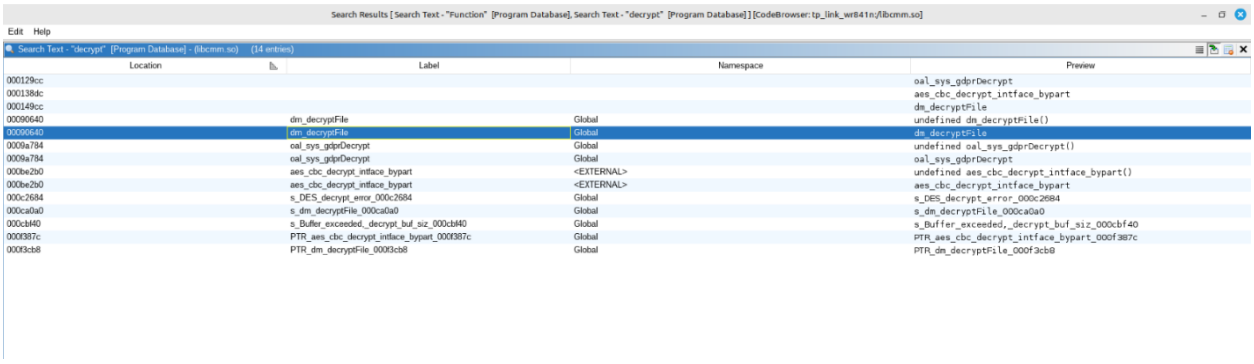


Figure 24 (b): function "dm_decryptFile"

In the function called "dm_decryptFile" we found the decompiled C code, Figure 25 (a). On the 5th line, the function "memcpy ()" is used, which revealed that the encryption algorithm is likely DES. Further investigation of the DES algorithm showed that it uses 64-bit (8 x 8 bytes) keys. At the beginning of the function, an array of 8 characters is created, and then memcpy copies 8 bytes into it from a data location in the .data section of the shared object file. Examining this data location revealed what appears to be the DES encryption key, refer to figure 25 (b). Since DES is a symmetric encryption algorithm, it can also be used for decryption.

```
int dm_decryptFile(uint param_1,undefined4 param_2,uint param_3,int param_4)

{
  int iVar1;
  undefined1 auStack_28 [8];
  int local_20;

  memcpy(auStack_28,&DAT_000c9f70,8);
  if (param_3 < param_1) {
    cdbg_printf(8,"dm_decryptFile",0xbcf,
                "Buffer exceeded, decrypt buf size is %u, but dm file size is %u",param_3,param_1); }(error message)
    local_20 = 0;
  }
  else {
    local_20 = cen_desMinDo(param_2,param_1,param_4,param_3,auStack_28,0);
    iVar1 = local_20;
    if (local_20 == 0) {
      cdbg_printf(8,"dm_decryptFile",0xbd6,"DES decrypt error\n"); }( This will let us know we're doing DES decrypt.)
    }
    else {
      do {
        local_20 = iVar1;
        if (((undefined1 *)(param_4 + local_20))[-1] != '\0') break;
        iVar1 = local_20 + -1;
      } while (local_20 != 0);
      *(undefined1 *)(param_4 + local_20) = 0;
    }
  }
  return local_20;
}
```

Figure 25(a): decompiled C code

```
                    DAT_000c9f70                                XREF[1]:     dm_decryptFile:00090688(*)
000c9f70 47          ??          47h    G
000c9f71 8d          ??          8Dh
000c9f72 a5          ??          A5h
000c9f73 0f          ??          0Fh
000c9f74 f9          ??          F9h
000c9f75 e3          ??          E3h
000c9f76 d2          ??          D2h
000c9f77 cb          ??          CBh
```

Figure 25(b): DES decryption Key

DES Encryption Key: 478DA50FF9E3D2CB

Now that we have the decryption key, we can use it to decrypt the default_config.xml file with OpenSSL, Figure 26. Simply open your terminal and run the following command:

openssl enc -d -des-ecb -K 478DA50FF9E3D2CB -nopad -in default_config.xml

```
lone@lone-HP-Laptop-15s-fr2xxx: ~/config
lone@lone-HP-Laptop-15s-fr2xxx:~$ cd config
lone@lone-HP-Laptop-15s-fr2xxx:~/config$ ls
default_config.xml  reduced_data_model.xml
lone@lone-HP-Laptop-15s-fr2xxx:~/config$ openssl enc -d -des-ecb -K 478DA50FF9E3D2CB -in default_config.xml
Error setting cipher DES-ECB
4077C37D8E740000:error:0308010C:digital envelope routines:inner_evp_generic_fetch:unsupported:../crypto/evp/evp_fetch.
c:386:Global default library context, Algorithm (DES-ECB : 1), Properties ()
lone@lone-HP-Laptop-15s-fr2xxx:~/config$ sudo openssl enc -d -des-ecb -K 478DA50FF9E3D2CB -in default_config.xml
<?xml version="1.0"?>
<DslCpeConfig>
  <InternetGatewayDevice>
    <DeviceSummary val="InternetGatewayDevice:1.1[](Baseline:1, EthernetLAN:1)" />
    <LANDeviceNumberOfEntries val=1 />
        <DeviceInfo>
        <X_TP_LogCfg>
                <ServerIP val=192.168.0.100 />
        </X_TP_LogCfg>
        </DeviceInfo>
    <X_TP_EthSwitch>
      <NumberOfVirtualPorts val=4 />
      <IfName val=eth0 />
    </X_TP_EthSwitch>
    <X_TP_NetCfg>
        <DNSServers val=0.0.0.0,0.0.0.0 />
        <DNSifAliasName val=ewan_ipoe_d />
    </X_TP_NetCfg>
    <X_TP_AppCfg>
                <PhDDNSCfg instance=1>
                <PhRTData>
                </PhRTData>
                </PhDDNSCfg>
                <PhDDNSCfg nextInstance=2 />
      <DynDnsCfg>
                        <Enable val=0 />
        <Server val=members.dyndns.org />
      </DynDnsCfg>
      <CmxDnsCfg>
                        <Enable val=0 />
                        <Server val=dns.comexe.cn />
      </CmxDnsCfg>
      <UPnPCfg>
        <Enable val=1 />
      </UPnPCfg>
                <NoipDnsCfg>
                        <Enable val=0 />
                        <Server val=dynupdate.no-ip.com />
                </NoipDnsCfg>
      <SnmpCfg>
        <Enable val=0 />
      </SnmpCfg>
      <ACL instance=1 >
        <Enable val=1 />
        <IPStart val=0.0.0.0 />
                <IPEnd val=0.0.0.0 />
                <Service val=Ping />
                <Location val=Local />
      </ACL>
```

Figure 26: Decrypted ".xml" with decryption Key

The enc command in OpenSSL is used to perform encryption or decryption operations.

- The -d option indicates that decryption is being performed.
- The -des-ecb specifies the use of the DES encryption algorithm in ECB mode.
- The -K option is followed by the decryption key in hexadecimal format.
- The -nopad option disables padding, assuming the original file was not padded.
- The -in option specifies the file to be decrypted.

After running this command, OpenSSL will decrypt the file using the provided key. If everything is correct, the decrypted content will be readable. This method is commonly used in security research or reverse engineering, especially when working with legacy encrypted configuration files. OpenSSL is a standard tool for such tasks.

Unfortunately, the config file did not contain any significant information.

Chapter 4

Differential Power Analysis on Cryptographic Processors

4.1 Does AES Encryption have any security problems?

Although AES is regarded as one of the most secure encryption techniques now in use, it is not impervious to attacks like any other security system. Researchers have discovered a few weaknesses over time. For example, a known-key attack on AES-128 was discovered in 2009, however, the practical risk was low because it only worked on an 8-round version of AES, not the complete 10-round one [27].

Side-channel attacks are a bigger issue for AES. Instead of directly attacking the encryption algorithm, these attacks try to obtain data from the equipment that is encrypting data, such as processing time, electromagnetic emissions, or even light and sound. Attackers can use this information to lessen the amount of work needed to use brute force to crack AES.

However, the pace at which AES can be cracked has only grown marginally, roughly three to five times faster, even with the most sophisticated assaults. Even yet, it would still take billions of years for supercomputers to crack AES completely. As a result, even if these results are significant, AES is still a reliable and safe encryption technique in the majority of real-world situations.[28].

4.2 Introduction to Power Analysis

The way a device consumes electricity can actually reveal what code it's executing. Different operations like adding versus multiplying numbers create distinctive power usage patterns that can be measured. These unique "power signatures" can be monitored and analyzed. Through this approach, we'll eventually explore how these power consumption patterns might be used to compromise cryptographic systems this is called Power Analysis [43].

Let's first examine the structure of an integrated circuit (IC) before speculating about its contents. Small parts make up an integrated circuit (IC), each of which is in charge of a distinct function. For example, one part may handle addition, while another may handle multiplication. Each of these parts generates a unique power pattern and operates at different times based on the task. This power consumption is essential to the execution of various attacks.

Let's look at a microcontroller that runs a bootloader that verifies a password, such as the ATMEL AVR. The chip stores your input in a temporary register (referred to as tmp1) when you enter the password. Next, it compares the two values character by character after retrieving the correct password from flash memory into a different register (tmp3).

The device goes into bootloader mode if the password is right. If it's incorrect, the gadget keeps looping without accomplishing anything. Our objective as attackers is to identify the moment the gadget enters this loop since it indicates the area in which our guess was wrong.

If you try alternative characters for the password's initial slot, for instance, the chip will enter the loop right away following a wrong guess. However, it indicates that the first character was accurate if it proceeds to check the second character. The complete password may be guessed by repeating this technique for every character. We call this a brute-force timing attack.

Certain timing assaults, such as a "wrong password" message or a blinking light, are simple to identify. But in this instance, the chip provides no clear input. So, how can we find out?

Power analysis becomes relevant in this situation. The chip's power usage during the password check allows us to determine whether it is actively comparing characters or looping. Power fluctuations can be visible even with slight timing discrepancies. The chip advances to the following character if the first one is accurate, and the power trace displays additional modifications. The chip rapidly loops back, and the power trace flattens out if the guess is incorrect. This lets you know if your guess was right without the chip providing you with any feedback.

It is possible for gadgets to unintentionally leak information. The power usage of a device can inadvertently expose private information when it executes code. We call this kind of information leak a side-channel. By monitoring how much power the device uses to process secret data, attackers can exploit these vulnerabilities and use that information to infer information about the contents.

We used a tool called ChipWhisperer, which simplifies the process [29]. Here’s how the setup works:

Power Measurement: To measure how much power the chip uses, we insert a small resistor (around 1–20 ohms) into its power line. We then measure the voltage drop across the resistor using a high-speed oscilloscope.

Triggering: We need to know when the device starts important operations, like comparing passwords or encrypting data. Often, a pin on the chip will turn on or off at this moment, and we use that as a trigger to start recording.

Data Collection: To detect patterns, we collect many power traces, sometimes hundreds or even thousands, while providing the chip with different inputs. This helps us identify small power variations that reveal what’s happening inside. This method is used in attacks like Differential Power Analysis (DPA).

4.3 Instruction on Power Differences

To see how a microcontroller responds to various kinds of programming, let's carry out a quick experiment. The STM32F3 [30] and XMEGA [30] are the two microcontrollers we'll be using, and we'll track how their power usage varies according to the jobs they do.

The many parts that make up each microcontroller are each in charge of distinct tasks. Certain components control memory, while others carry out computations and handle inputs and outputs.

The chip consumes less power when performing simple operations like reading data or simple calculations. However, more energy is needed for more complicated operations like encryption or processing massive volumes of data.

We utilize a tool called ChipWhisperer, which is compatible with the OPENADC module [31], to witness this. These instruments enable us to gauge the chip's power usage while it works. To inform the chip what to do, we first load some code onto it. Tools such as ST-Link for the STM32F3 and PDI (Program and Debug Interface) for the XMEGA are used to upload this code.

The microcontroller and ChipWhisperer are then connected. Sometimes the connection might not work the first time, but we just reconnect until it does.

The ChipWhisperer is then notified to begin recording when the chip starts its work using a trigger signal, which is a GPIO pin. By doing this, we can only gauge power use during significant occasions.

Our tools record the power consumption of the chip as it is executing its code. On the graph, a basic task produces one pattern, but a more difficult task produces a different pattern. We record power traces for particular actions as well as idle power traces.

For idle power traces, refer to Figure 27 (a) for the STM32 F303 microcontroller and Figure 27 (b) for the XMEGA microcontroller.

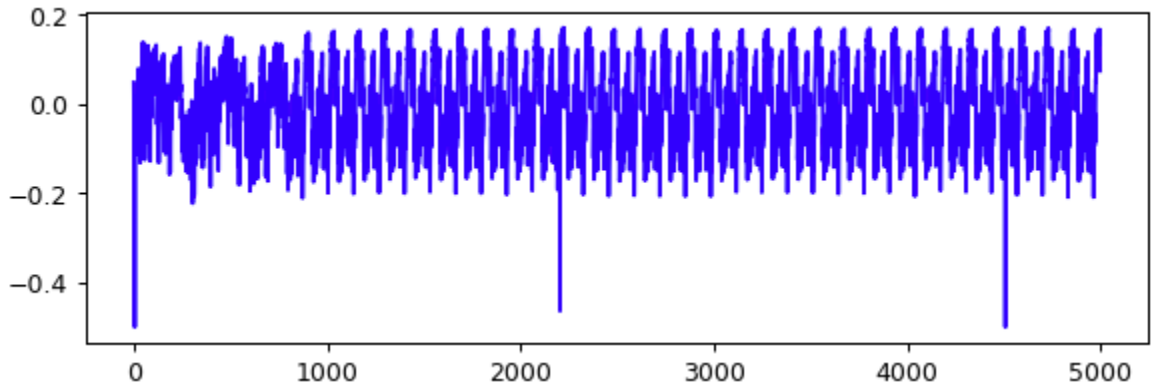


Figure 27 (a): Idle Power trace of STM32 F303

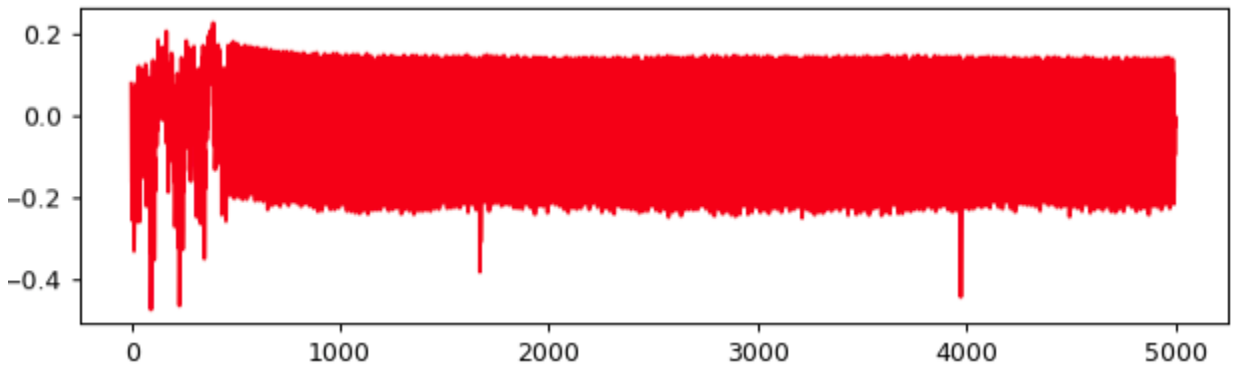


Figure 27 (b): Idle Power trace of ATMEL XMEGA

For operational power traces, see Figure 28(a) for the STM32 F303 and Figure 28(b) for the XMEGA.

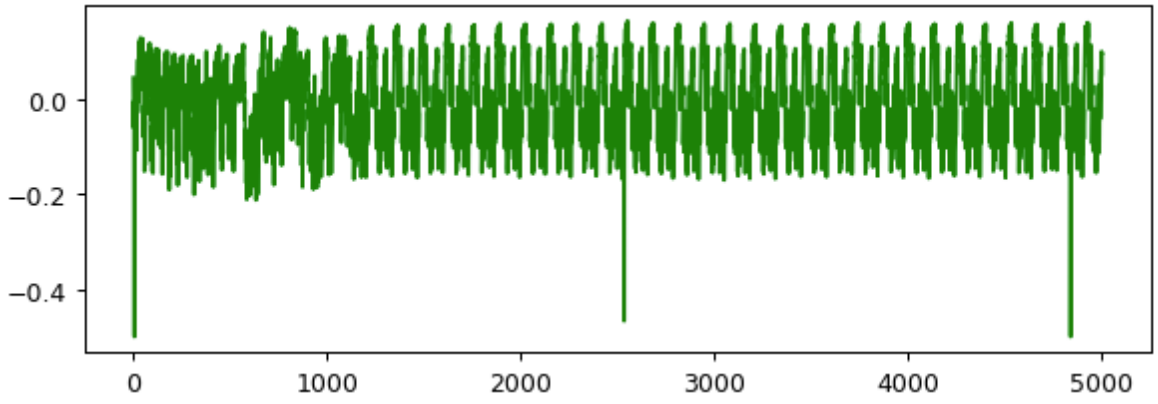


Figure 28 (a): Operational Power Traces of STM32 F303

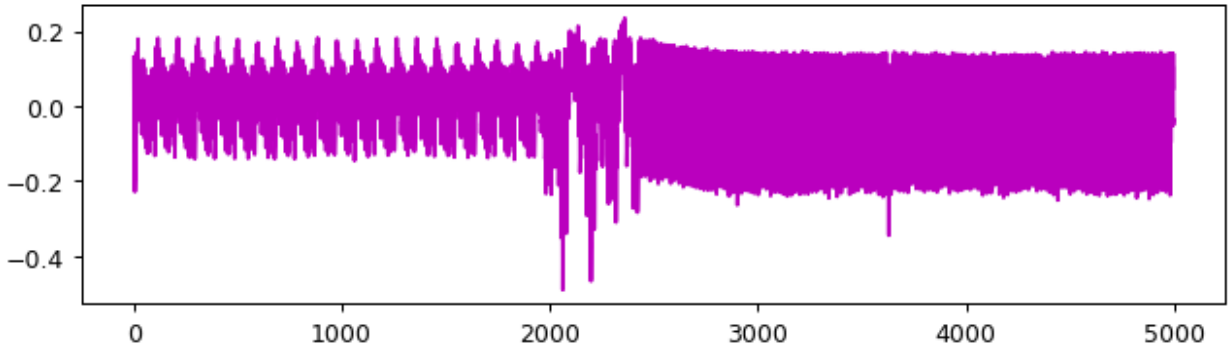


Figure 28 (b): Operational Power Traces of ATMEL XMEGA

4.4 Differential Power Analysis

This attack exploits the fact that little differences in power consumption result from changing data values processed by a microprocessor. By employing statistical methods to examine the patterns of power usage over time, these variations can be found.

Take the Advanced Encryption Standard, or AES, for instance. Following an XOR between a byte of the plaintext and a section of the round key, the chip does a substitution operation known as an S-box in the first round. Now consider an attacker who records several power traces as the device encrypts different plaintexts. For every key byte guess, the attacker concentrates on a single bit, like the least significant bit (LSB), from the S-box output. After that, they divide the power traces into two groups: one for which the bit is 0 and another for which it is 1.

The average power consumption in the "0" and "1" sets will be significantly different if the key guess is accurate. Known as the "difference of means," this disparity will show up as a spike on the graph. The "difference of means," which manifests as a spike, is visible in Figures 29(a) with 29(b).

In Figure 29(a), we see the differential power analysis of a single key bit for the XMEGA, while in Figure 29(b), we see the differential power analysis of a single key bit for the STM32 F303.

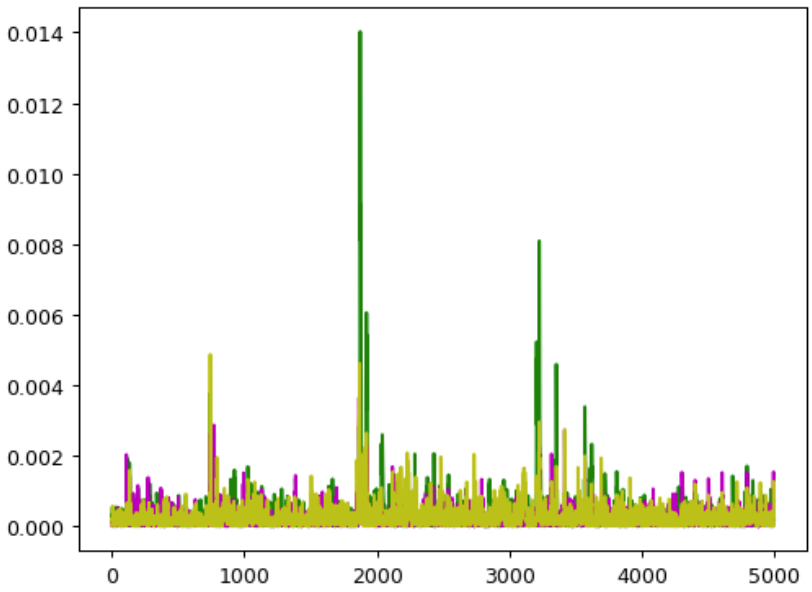


Figure 29 (a): DPA of a single key bit for the XMEGA

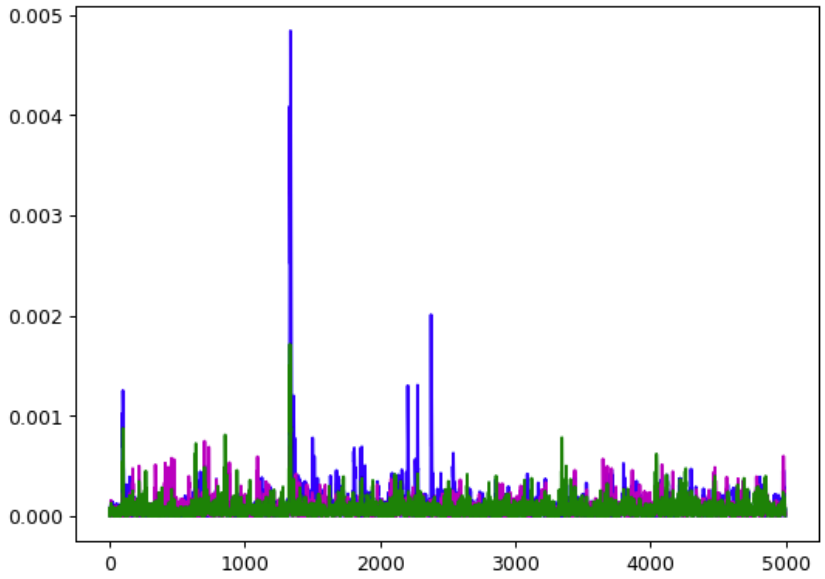


Figure 29 (b): DPA of a single key bit for the STM32 F303

With its software and scripts, you can replicate this attack on AES, examine different S-box bits, and test countermeasures such as masking or power balancing. Figure 30 shows the cracked AES key.

Cracked AES key: 2b7e151628aed2a6abf7158809cf4f3c

```
100% ■■■
```

Chapter 5

Conclusion and Future Work

This thesis demonstrates that firmware key extraction is not only possible but also highly effective using side-channel analysis techniques like Differential Power Analysis (DPA). The experiments confirmed that even well-secured embedded systems can be vulnerable if attackers have physical access and the right measurement tools. DPA remains a significant threat, mainly because it is difficult to detect and often doesn't leave obvious evidence.

Looking ahead, embedded security will face even greater challenges. One key area is post-quantum cryptography, which seeks to protect systems from quantum computer attacks. As quantum computing advances, traditional encryption methods may no longer be sufficient. Additionally, more robust hardware designs are needed to resist side-channel attacks by default, such as built-in noise generators, balanced power consumption, and masking techniques.

Another promising direction is the use of automation and machine learning to detect and respond to side-channel attacks more quickly and accurately. Automating detection can improve real-time defense and reduce human errors. Overall, while current attacks are powerful, stronger and more advanced defenses are within reach.

References

[1] Cybersecurity and Infrastructure Security Agency (CISA), “Advanced Persistent Threat Compromise of Government Agencies, Critical Infrastructure, and Private Sector Organizations,” Cybersecurity Advisory AA20-352A, Dec. 17, 2020. [Online]. Available: https://www.cisa.gov/news-events/cybersecurity-advisories/aa20-352a . [Accessed: May 11, 2025].

[2] U.S. Government Accountability Office, “Actions Taken by Equifax and Federal Agencies in Response to the 2017 Breach,” GAO-18-559, 2018. [Online]. Available: https://www.gao.gov/products/gao-18-559. [Accessed: May 11, 2025].

[3] European Union Agency for Cybersecurity (ENISA), “ENISA Threat Landscape 2023,” Oct. 2023. [Online]. Available: https://www.enisa.europa.eu/publications/enisa-threat-landscape-2023 . [Accessed: May 13, 2025].

[4] Tripwire, "NotPetya: Timeline of a Ransomworm," *Tripwire State of Security*, 2017. [Online]. Available:https://www.tripwire.com/state-of-security/notpetya-timeline-of-a-ransomworm . [Accessed: May 11, 2025].

[5] R. Langner, “Stuxnet: Dissecting a cyberwarfare weapon,” *IEEE Security & Privacy*, vol. 9, no. 3, pp. 49–51, 2011.

[6] Moritz Lipp, Michael Schwarz, Daniel Gruss, Thomas Prescher, Werner Haas, Anders Fogh, Jann Horn, Stefan Mangard, Paul Kocher, Daniel Genkin, Yuval Yarom, and Mike Hamburg. 2018. Meltdown: reading kernel memory from user space. In Proceedings of the 27th USENIX Conference on Security Symposium (SEC'18). USENIX Association, USA, 973–990.

[7] P. Kocher et al., “Spectre attacks: Exploiting speculative execution,” in *Proc. IEEE Symposium on Security and Privacy*, 2019, pp. 1–19.

[8] W. Stallings, *Network Security Essentials: Applications and Standards*, 6th ed., Pearson, 2017.

[9] Y. Xiao et al., “Survey of hardware security techniques,” *IEEE Design & Test*, vol. 36, no. 6, pp. 17–30, 2019.

[10] Leyla Bilge and Tudor Dumitraş. 2012. Before we knew it: an empirical study of zero-day attacks in the real world. In Proceedings of the 2012 ACM conference on Computer and communications security (CCS '12). Association for Computing Machinery, New York, NY, USA, 833–844. https://doi.org/10.1145/2382196.2382284 .

[11] P. Ning and A. Liu, “Challenges in securing the Internet of Things,” *IEEE Internet of Things Journal*, vol. 3, no. 4, pp. 472–475, 2016.

[12] Ross Anderson and Markus Kuhn. 1996. Tamper resistance: a cautionary note. In Proceedings of the 2nd conference on Proceedings of the Second USENIX Workshop on Electronic Commerce - Volume 2 (WOEC'96). USENIX Association, USA, 1.

[13] D. Holcomb, W. Burleson, and K. Fu, “Power-up SRAM state as an identifying fingerprint and source of true random numbers,” *IEEE Transactions on Computers*, vol. 58, no. 9, pp. 1198–1210, 2009.

[14] Trusted Computing Group, “TPM main specification,” Version 1.2, 2011. [Online]. Available: https://trustedcomputinggroup.org/resource/tpm-main-specification/ . [Accessed: May 11, 2025].

[15] IBM, “Hardware security module overview,” IBM Security, 2020. [Online]. Available: https://www.ibm.com/docs/en/hsma/overview . [Accessed: May 11, 2025].

[16] Schneier, B. (1993). *Applied cryptography: Protocols, algorithms, and source code in C*. John Wiley & Sons, Inc.

[17] W. Diffie and M. Hellman, “New directions in cryptography,” *IEEE Transactions on Information Theory*, vol. 22, no. 6, pp. 644–654, 1976.

[18] National Institute of Standards and Technology (NIST), “FIPS PUB 197: Advanced Encryption Standard (AES),” Nov. 2001. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf . [Accessed: May 11, 2025].

[19] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 8th ed., Pearson, 2023.

[20] M. E. Smid and D. K. Branstad, "Data Encryption Standard: past and future," in *Proceedings of the IEEE*, vol. 76, no. 5, pp. 550-559, May 1988, doi: 10.1109/5.4441.

[21] E. Rescorla, “The Death of DES,” IEEE Internet Computing, vol. 6, no. 6, pp. 86–89, Nov.-Dec. 2002.

[22] N. Otterness, "Tiny ELF Files: Revisited in 2021," *nathanotterness.com*, Oct. 11, 2021. [Online]. Available: https://nathanotterness.com/2021/10/tiny_elf_modernized.html . [Accessed: May 11, 2025].

[23] P. Dhaker, "Introduction to SPI Interface," *Analog Devices*, Oct. 11, 2021. [Online]. Available: https://www.analog.com/en/resources/analog-dialogue/articles/introduction-to-spi-interface.html. [Accessed: May 11, 2025].

[24] Sigrok Project, “PulseView - The Sigrok GUI,” [Online]. Available: https://sigrok.org/wiki/PulseView

[25] C. Wright, "Binwalk: Firmware Analysis Tool," ReFirm Labs, [Online]. Available: https://github.com/ReFirmLabs/binwalk.

[26] N. S. A., Ghidra Software Reverse Engineering Framework, 2019. [Online]. Available: https://ghidra-sre.org

[27] M. Gorski and S. Lucks, "New Related-Key Boomerang Attacks on AES," IACR Cryptology ePrint Archive, 2008. [Online]. Available: https://eprint.iacr.org/2008/438.pdf . [Accessed: May 11, 2025].

[28] National Institute of Standards and Technology (NIST), “FIPS PUB 197: Advanced Encryption Standard (AES),” Nov. 2001. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf . [Accessed: May 11, 2025].

[29] C. O’Flynn, “A Tutorial on Side Channel Power Analysis Using the ChipWhisperer,” in IACR Cryptology ePrint Archive, 2014.

[30] NewAE Technology Inc., "Targets," *ChipWhisperer Documentation*, 2025. [Online]. Available: https://chipwhisperer.readthedocs.io/en/latest/Targets/index.html. [Accessed: May 11, 2025].

[31] NewAE Technology Inc., "OpenADC.py," *ChipWhisperer GitHub Repository*, 2025. [Online]. Available:https://github.com/newaetech/chipwhisperer/blob/develop/software/chipwhisperer/capture/scopes/OpenADC.py. [Accessed: May 11, 2025].

[32] National Security Agency (NSA), "Four years later: The impacts of Ghidra's public release," [Online]. Available: https://www.nsa.gov/Press-Room/News-Highlights/Article/Article/3319971/four-years-later-the-impacts-of-ghidras-public-release/ . [Accessed: May 11, 2025].

[33] Geekstory, "CH341A 24/25 Series EEPROM Flash BIOS USB Programmer Module + SOIC8 Test Clip," Amazon, [Online]. Available: https://www.amazon.com/Geekstory-CH341A-EEPROM-Programmer-Module/dp/B098DYJ3LQ . [Accessed: May 11, 2025].

[34] TP-Link, "TL-WR841N: 300Mbps Wireless N Router," [Online]. Available: https://www.tp-link.com/us/home-networking/wifi-router/tl-wr841n . [Accessed: May 11, 2025].

[35] Analog Devices, "Introduction to SPI Interface," Analog Dialogue, [Online]. Available: https://www.analog.com/en/resources/analog-dialogue/articles/introduction-to-spi-interface.html . [Accessed: May 11, 2025].

[36] UnicornSecurity, "Reversing TL-WR840N," [Online]. Available: https://therealunicornsecurity.github.io/TPLink/ . [Accessed: May 11, 2025].

[37] ZenArmor, "What is the Advanced Encryption Standard (AES)? – MixColumns," [Online]. Available: https://www.zenarmor.com/docs/network-security-tutorials/what-is-advanced-encryption-standard-aes . [Accessed: May 11, 2025].

[38] ZenArmor, "What is the Advanced Encryption Standard (AES)? – ShiftRows," [Online]. Available: https://www.zenarmor.com/docs/network-security-tutorials/what-is-advanced-encryption-standard-aes. [Accessed: May 11, 2025].

[39] ZenArmor, "What is the Advanced Encryption Standard (AES)? – SubBytes," [Online]. Available: https://www.zenarmor.com/docs/network-security-tutorials/what-is-advanced-encryption-standard-aes. [Accessed: May 11, 2025].

[40] Braincoke, "The AES encryption algorithm explained," Aug. 2020. [Online]. Available: https://braincoke.fr/blog/2020/08/the-aes-encryption-algorithm-explained. [Accessed: May 11, 2025].

[41] ZenArmor, "What is Data Encryption Standard (DES) in Cryptography?", [Online]. Available: https://www.zenarmor.com/docs/network-security-tutorials/what-is-data-encryption-standard-des. [Accessed: May 11, 2025].

[42] SSL2BUY, "Symmetric vs. Asymmetric Encryption – What Are the Differences?", [Online]. Available: https://www.ssl2buy.com/wiki/symmetric-vs-asymmetric-encryption-what-are-differences . [Accessed: May 11, 2025].

[43] P. Kocher, J. Jaffe, and B. Jun, “Differential Power Analysis,” in *Proc. 19th Annual International Cryptology Conference (CRYPTO)*, Santa Barbara, CA, USA, Aug. 1999, pp. 388–397.